Projet 10 — Rapport de mise en place et d'évaluation du système RAG SportSee

Cheun Pelleter · octobre 2026

Version livrée : v2b. Sur les 50 questions du jeu principal, elle donne 11 réponses simples exactes sur 12, 10 multicritères sur 12 et 10 refus corrects sur 10, sans erreur technique, contre 0, 4 et 0 pour le prototype. Son point faible est le jeu de robustesse mixte (texte Reddit et chiffres) : 2 réponses correctes sur 8. Des outils de calcul déterministes ont porté ce score jusqu'à 6 sur 8 (v2d), mais en faisant régresser les multicritères (8 sur 12 au mieux) : selon le critère fixé avant l'évaluation, ils ne sont pas livrés et restent la piste prioritaire.

1. Contexte et objectifs

SportSee veut un assistant qui réponde aux coachs et analystes sur la saison régulière NBA 2024-25, à partir de deux sources : un fichier Excel de statistiques (569 joueurs, totaux de saison) et quatre PDF de discussions Reddit.

Le point de départ. Un prototype RAG existait (Mistral, index FAISS, interface Streamlit), sans aucune mesure de sa fiabilité. Les coachs signalaient des chiffres faux, sans moyen d'en mesurer l'ampleur. Première mesure : 0 question simple exacte sur 12, 0 refus correct sur 10.

L'objectif. Mettre en place un système évalué de façon mesurable et reproductible, l'améliorer, puis prouver chaque amélioration sur le même jeu de test.

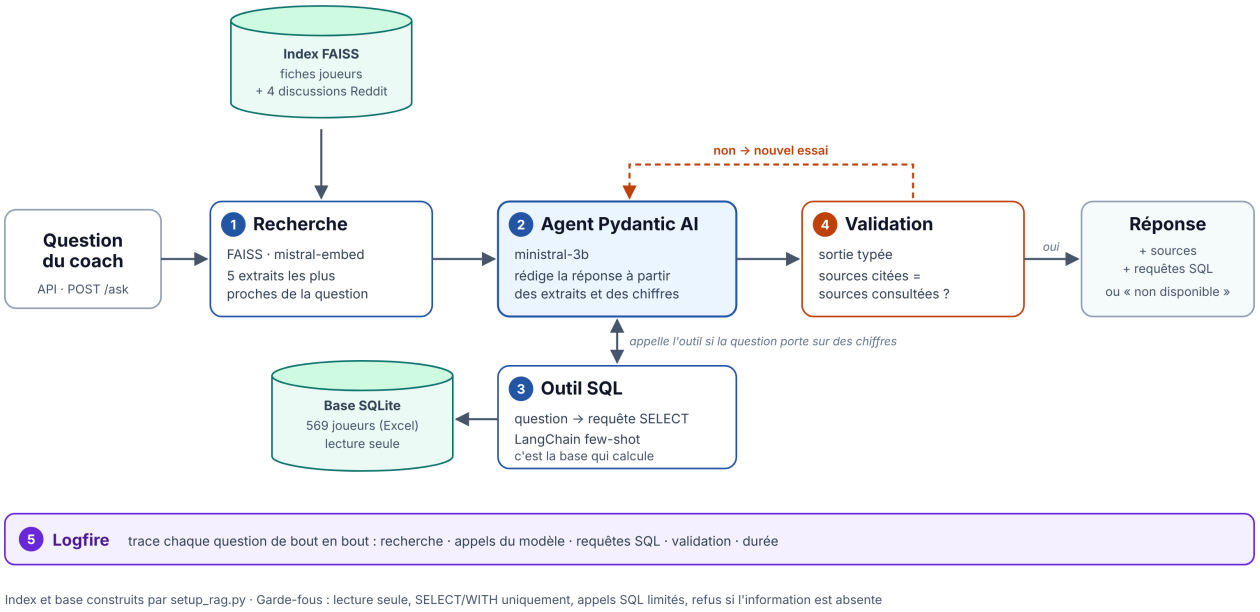
La démarche, en trois étapes :

1. **Évaluation de référence et préparation des données** : jeu de 50 questions métier, évaluation RAGAS, données validées par Pydantic, agent Pydantic AI, suivi Logfire (versions v0 à v1c).
2. **Base SQL et outil d'interrogation** : base SQLite chargée par un script validé par Pydantic, outil SQL (LangChain, exemples few-shot) branché sur l'agent (v2, v2b).
3. **Réévaluation, robustesse et analyse critique** : comparaison des versions, jeu de questions mixtes (texte et chiffres), outils de calcul déterministes (v2c à v2e), mise en place reproductible (script unique, API, journalisation).

Chaque version ne change qu'une famille de variables. Le générateur (`ministral-3b-2512`), le juge (`ministral-14b-2512`), le jeu de test et l'index vectoriel restent identiques d'une version à l'autre.

2. Architecture et choix techniques

Le système combine deux sources : un index vectoriel FAISS pour les discussions et le contexte, une base SQLite pour les chiffres. Un agent Pydantic AI choisit ses outils, et sa réponse est validée avant d'être rendue.



Architecture du système : recherche FAISS, agent Pydantic AI, outil SQL, validation des sources, traces Logfire

Hors ligne (`setup_rag.py`), les PDF sont lus, découpés en 644 extraits et vectorisés ; l'Excel est validé ligne par ligne par Pydantic puis chargé dans SQLite. **En ligne**, la recherche FAISS est toujours faite (5 extraits) ; l'agent appelle un outil de données quand la question est chiffrée, puis sa réponse passe par le validateur de sources.

Dans la version livrée (v2b), l'agent dispose de l'outil `interroger_statistiques`. Les deux outils déterministes, développés en v2d et v2e, sont dans le dépôt et testés, mais non branchés sur l'agent (section 6). Limite stricte de 2 appels par question :

Outil	Le modèle fournit	Le code fait	Pour quelles questions
`interroger_statistiques` (v2)	une question en français	génère le SQL (LangChain, 9 exemples), le vérifie, l'exécute en lecture seule	classements, filtres, moyennes, « qui a le plus… »
`rang_joueur` (v2d)	statistique, joueur	SQL écrit à l'avance : classe toute la ligue, puis repère le joueur	rang d'un joueur nommé, « est-ce le meilleur ? »
`comparer_a_un_seuil` (v2d)	statistique, joueur, seuil	compare et renvoie la conclusion en clair	« a-t-il dépassé 35 % ? »

2.1 Choix techniques

Élément	Choix	Raison
Générateur	`ministral-3b-2512`	Seul modèle rapide accessible sans limite : `mistral-large` refusé par l'abonnement (403), `small` et `medium` limités en débit (429), mesuré par `scripts/verifier_acces_modeles.py`

Élément	Choix	Raison
Juge RAGAS	`ministral-14b-2512`	Plus grand que le modèle évalué (bonne pratique), accessible sans limite
Embeddings	`mistral-embed` (1 024 dimensions)	Même fournisseur, cohérent avec le générateur
Retriever	FAISS exact, similarité cosinus, 5 extraits	Exact et sans serveur pour 644 extraits ; pas besoin d'index approché à cette taille
Base	SQLite	Un fichier, rien à installer, reproductible
Agent	Pydantic AI	Sortie typée (`ReponseAssistant`), validation et relances intégrées, outils typés
Outil SQL génératif	LangChain, 9 exemples few-shot	Demandé par l'énoncé ; un petit modèle imite mieux qu'il n'applique des règles
Outils déterministes	SQL fixe, paramètres typés (`Literal`)	Le modèle comprend la question, le code calcule (section 7.2)
API	FastAPI, schémas Pydantic	Validation des entrées, documentation `/docs` générée
Observabilité	Logfire	Trace de chaque étape, requêtes SQL comprises

2.2 Les garde-fous

- **Base en lecture seule** et contrôle applicatif du SQL généré : une seule instruction `SELECT` ou `WITH`, mots-clés d'écriture refusés, nombre de lignes borné.
- **Outils déterministes** : seule une colonne de la liste fermée (42 colonnes) est insérée dans le SQL ; le joueur et le seuil passent en paramètres liés.
- **Validateur de sources** (`ReponseAssistant`) : une source citée doit figurer parmi les extraits récupérés ou la base ; un refus ne cite aucune source ; des chiffres donnés exigent une source.
- **Limite d'appels stricte**, protégée par un verrou contre les appels parallèles (section 7.2).

3. Mise en place du système

Un seul lancement, `uv run setup_rag.py`, rend le système opérationnel en moins de 20 secondes quand l'index existe : il vérifie l'environnement, construit l'index et la base, puis pose deux questions de contrôle.

3.1 Le script de mise en place

Étape	Rôle	Durée mesurée
`environnement`	clé API et fichiers d'entrée vérifiés avant tout traitement long	1 à 2 s
`index`	index FAISS construit (OCR + embeddings), ou réutilisé s'il existe	0 s (réutilisé)

Étape	Rôle	Durée mesurée
`base`	`data/nba.db` reconstruite à partir de l'Excel validé, comptes contrôlés par relecture	1,5 à 3 s
`test`	une question à laquelle répondre (3P% de Harden : 35,2 %), une à refuser (Roland-Garros)	3 à 7 s, plus le chargement des bibliothèques (environ 15 s au total)

Chaque étape est une fonction journalisée avec sa durée ; le script s'arrête à la première étape en échec et renvoie un code de sortie (0 ou 1), utilisable par une intégration continue. Les options `--etapes` et `--force-index` permettent de rejouer une partie.

3.2 L'API

L'API FastAPI (`api.py`) expose `POST /ask` et `GET /health`, documentées automatiquement sur `/docs`. Les entrées et sorties sont des modèles Pydantic : une question vide ou de plus de 500 caractères est refusée (422) avant tout appel au modèle. Un refus métier reste un 200 avec `information_disponible: false` ; une panne du modèle renvoie 503. La réponse expose les sources citées et la trace des requêtes SQL, pour la transparence envers les coachs. Temps de réponse mesuré : 0,8 s (lecture d'une fiche) à 2,5 s (avec requête SQL).

3.3 Journalisation et observabilité

- **Journal structuré** (`horodatage | niveau | module | message`), configuré uniquement par les points d'entrée ; chaque module écrit sous son nom. Fichiers : `logs/setup_rag.log`, `eval/results/<version>/evaluation.log`.
- **Logfire** trace chaque question : recherche (sources, scores), construction du prompt, appels du modèle, appels d'outils avec leurs paramètres et requêtes, bilan de la réponse.
- **Durées** : par étape de mise en place, par question (API et évaluation), et pour la notation RAGAS (environ 30 minutes pour 50 questions).

3.4 Dépendances et organisation du dépôt

Les dépendances sont déclarées dans `pyproject.toml` (uv) et figées dans `uv.lock` et `requirements.txt`. Elles sont séparées en groupes : l'assistant et l'API tournent avec les seules dépendances principales, sans PyTorch ni RAGAS (82 paquets en moins, vérifié par une installation réduite) ; `indexing` (OCR) et `eval` (RAGAS) s'ajoutent si besoin.

Emplacement	Contenu
racine	les commandes du README : `setup_rag.py`, `api.py`, `evaluate_ragas.py`, `rag_pipeline.py`, `sql_tool.py`, `load_excel_to_db.py`, `indexer.py`
`utils/`	code partagé : schémas Pydantic, schéma SQL, outils de l'agent, exactitude, seuils
`scripts/`	outils d'analyse : comparaison de versions, refus, requêtes SQL, diagnostics

Emplacement	Contenu
`tests_rag/`	tests déterministes des outils de classement, sans appel à l'API
`eval/`	jeux de test, éléments attendus, résultats de chaque version
`docs/`	rapport, journal des anomalies de données

3.5 Tests

`tests_rag/test_outils_classement.py` vérifie sur la vraie base, sans appel à l'API : rangs calculés sur toute la ligue (SGA 2e aux lancers francs tentés, Cunningham 3e aux balles perdues), comparaisons à un seuil, joueur ambigu (« Wagner ») ou absent (« Reggie Miller »), noms abîmés des fiches (« P,J, Washington »), limite d'appels stricte sur 10 appels parallèles. Les doctests de `utils/nettoyage_donnees.py` couvrent le nettoyage des noms.

4. Qualité des données

Les 569 joueurs passent la validation sans rejet, mais l'Excel contient des anomalies cachées : 31 noms abîmés, 206 totaux de rebonds incohérents, des pourcentages qui ne correspondent pas aux tirs réussis et tentés, et un dictionnaire des données faux sur les unités.

4.1 Validation par Pydantic

Chaque ligne passe par un modèle Pydantic avant d'être indexée ou insérée en base :

- **`PlayerStats`** (45 colonnes) : types et bornes, alias vers les noms de l'Excel, nettoyage du nom avant validation, contrôle de l'équipe par un contexte de validation (les codes de la feuille « Equipe »).
- **`Team`** : code de trois majuscules, nom complet.
- **`Chunk`** et **`EmbeddedChunk`** : longueur des extraits, dimension (1 024) et non-nullité des vecteurs.
- **`ReponseAssistant`** : la sortie de l'agent, avec un validateur de cohérence des sources.

Les règles **bloquantes** rejettent une ligne impossible (`W + L ≠ GP`, plus de tirs réussis que tentés, plus de triple-doubles que de double-doubles). Les règles **non bloquantes** signalent un écart sans rejeter la ligne. Bilan du chargement : 0 rejet, 31 corrections, 1 458 anomalies conservées et signalées.

4.2 Anomalies et décisions

Le principe : **corriger ce qui est certain, signaler ce qui ne l'est pas, ne jamais modifier le fichier source.** Les corrections vivent dans le code, donc elles sont tracées et rejouées à chaque chargement.

Anomalie	Lignes	Traitement	Justification
Noms avec virgules (`Jaren Jackson Jr,`, `P,J, Washington`)	31	Corrigé (virgule → point)	Aucun nom NBA ne contient de virgule : correction certaine

Anomalie	Lignes	Traitement	Justification
`REB ≠ OREB + DREB` (écarts de −8 à +8)	206	Conservé et signalé	On ignore quelle valeur est fausse ; le jeu de test utilise la colonne REB
FG%, 3P%, FT% différents de réussis / tentés	environ 1 250	Signalé	Même raisonnement ; les références viennent de ces colonnes
EFG% et TS% supérieurs à 100	quelques-unes	Accepté (borne à 150)	Possible sur très peu de tirs
Colonne 3PM lue comme l'heure 15:00:00	1 colonne	Renommée (repérée par son type)	Erreur de conversion d'Excel
Dictionnaire : PTS, FGM… décrits « par match »	—	Unités réécrites pour le LLM	Ce sont des totaux (SGA : 2 485 points)
Aucune donnée par match	—	Table `matches` créée vide	Rien n'est inventé ; le schéma reste extensible

Toutes les anomalies sont consignées dans `docs/audit/anomalies_donnees.csv` (joueur, règle, détail, traitement).

4.3 La base SQL

La base SQLite (`data/nba.db`) compte quatre tables et une vue : `teams` (30) et `players` (569), reliées par clé étrangère ; `player_stats` (569 lignes, une par joueur), dont les colonnes sont **générées à partir du modèle Pydantic**, de sorte que modèle et table ne peuvent pas diverger ; `matches`, vide ; et la vue `player_per_game`, qui calcule les moyennes par match. C'est la base, et non le LLM, qui divise. Chaque colonne est décrite avec son unité : c'est ce texte que lit le modèle qui écrit le SQL.

5. Méthodologie d'évaluation

L'évaluation combine RAGAS, une exactitude déterministe et une relecture manuelle, car RAGAS seul notait favorablement des réponses fausses sur un chiffre.

5.1 Les jeux de test et leur justification métier

Deux jeux, séparés : le **jeu principal** (50 questions, figé depuis l'étape 1, il sert à comparer toutes les versions) et le **jeu de robustesse mixte** (8 questions, ajouté à l'étape 3). Chaque catégorie correspond à un risque concret pour un coach :

Catégorie	Questions	Ce qu'elle teste	Pourquoi c'est un risque métier
Simple	12	Une valeur d'un joueur (3P% de Booker)	Usage le plus fréquent ; un chiffre faux fausse une décision de jeu

Catégorie	Questions	Ce qu'elle teste	Pourquoi c'est un risque métier
Multicritères	12	Classement, filtre, comparaison sur 569 joueurs	Le cas où le prototype échouait : 5 fiches ne suffisent pas à classer la ligue
Bruitée	8	Fautes, formulations familières (variantes de questions simples)	Un coach tape vite, sur une tablette, au bord du terrain
Textuelle	8	Contenu des discussions Reddit	L'avis des fans et des analystes, que la base ne contient pas
Hors données	6	Information absente (playoffs, salaire, 5 derniers matchs)	Inventer une réponse est le défaut le plus grave d'un outil d'aide à la décision
Hors sujet	4	Question sans rapport (Roland-Garros)	L'assistant doit connaître son périmètre
Mixte	**8**	**Une information de Reddit et un chiffre ou un rang de la base**	**Vérifier une affirmation lue sur un forum avec les données : « est-ce vrai qu'il a dépassé 35 % ? »**

Le jeu mixte comprend aussi des pièges : une période différente (45 % en playoffs selon Reddit, 34,4 % en saison régulière), des prénoms seuls (« Paolo et Franz », alors qu'il y a deux Wagner à Orlando), des rangs à calculer, et un joueur absent dont le nom ressemble à d'autres (Reggie Miller, face à Reggie Jackson et Brandon Miller). Toutes les références chiffrées sont calculées avec pandas à partir de l'Excel, jamais écrites à la main.

5.2 Les métriques

RAGAS (0.4.3), juge `ministral-14b-2512` : `faithfulness` (réponse fondée sur le contexte), `answer_relevancy` (réponse à la question), `context_precision` et `context_recall` (qualité de la recherche), `answer_correctness` (proximité avec la référence).

Les métriques complémentaires, ajoutées parce que RAGAS s'est révélé insuffisant :

- **Exactitude déterministe** (`utils/exactitude.py`) : la réponse contient-elle les éléments attendus (nombres à 0,05 près, noms sans accents ni casse) ? Créée après avoir vu `answer_correctness` noter 0,75 une réponse fausse (9,8 au lieu de 8,5).
- **Taux de refus** sur les 10 questions sans réponse possible, déclaré par l'agent puis **vérifié à la main**.
- **Réponses en erreur** : pannes techniques ou validations échouées.
- **Relecture manuelle** des questions mixtes, nécessaire car l'exactitude ne vérifie pas les conclusions (section 7.4).

5.3 Les seuils

Aucun seuil n'a été fixé avant la v0, volontairement. Les premières versions (v0, v1) formaient une phase exploratoire : il fallait savoir ce que le système faisait et ce que mesuraient vraiment les métriques. Cette phase a montré par exemple qu'`answer_correctness` notait 0,75 une réponse fausse, et a conduit à créer l'exactitude. Les seuils ont été fixés après la v1, **figés avant la v2** et plus modifiés ensuite. Ceux du jeu mixte ont été fixés **avant toute évaluation** de ce jeu.

Catégorie	Métrique	Bêta	Cible	Justification
Toutes (à réponse)	faithfulness	0,80	0,85	Un chiffre non fondé sur les données peut fausser une décision
Simple, bruitée	exactitude	0,80	0,90	Une valeur unique lue dans les données : l'erreur n'est pas admissible
Multicritères, mixte	exactitude	0,60	0,80	Calcul, classement ou deux sources : tâche plus difficile
Simple, bruitée, multicritères, mixte	answer_correctness	0,60	0,75	Indicateur RAGAS de justesse, complémentaire
Textuelle	answer_correctness	0,50	0,60	Réponses en langage libre, jugées par proximité
Hors données, hors sujet	taux de refus	100 %	100 %	Inventer est le défaut le plus grave
Toutes	réponses en erreur	0 %	0 %	Une panne visible dégrade la confiance

Bêta signifie « ouvrable aux coachs sous surveillance », cible « production ». Limite assumée : les seuils du jeu principal ont été calibrés après avoir vu la v0 et la v1.

5.4 La rigueur du protocole

- **Une famille de variables à la fois** : données (v1a), génération (v1b), validation (v1c), outil SQL (v2), correctifs (v2b), prompt (v2c), outils déterministes (v2d), fiabilisation (v2e).
- **Constantes** : même jeu de test, même juge, même générateur, même index depuis la v1a.
- **Mesure du bruit** : la v0 a été évaluée deux fois.
- **Comparaison à questions égales** (`scripts/comparer_versions.py`).
- **Traçabilité** : réponses, contextes, requêtes SQL et description du pipeline enregistrés pour chaque évaluation ; un tag Git par version livrée ou essai (`v1b`, `v2b`, `v2c-essai`…).
- **Diagnostic avant correction** : chaque échec est classé en relisant la réponse et la requête exécutée (`scripts/voir_requetes_sql.py`).

6. Résultats

La v2b, version livrée, porte les questions multicritères de 1 à 10 exactes sur 12 grâce à l'outil SQL. Les outils déterministes (v2d, v2e) améliorent les questions mixtes (2 sur 8 → 6 puis 4) mais font régresser les multicritères : ils ne sont pas livrés.

6.1 Le jeu principal (50 questions)

Version	Changement	Simples exactes	Bruitées	Multicritères	Refus corrects	Erreurs
v0	Prototype d'origine	0/12	2/8	4/12	0/10	0
v1a	Données restructurées (une fiche par joueur)	10/12	3/8	1/12	0/10	0
v1b	Prompt pour coachs, agent Pydantic AI, Logfire	10/12	4/8	1/12	10/10	0
v2	Outil SQL (SQLite, LangChain few-shot)	11/12	5/8	8/12	9/10	3
v2b (livrée)	**Correctifs, règles 13-14**	**11/12**	**5/8**	**10/12**	**10/10**	**0**
v2d	Outils déterministes `rang_joueur`, `comparer_a_un_seuil`	12/12	4/8	7/12	10/10	2
v2e	v2d fiabilisée : règle 15 restreinte, limite d'appels stricte, noms nettoyés	12/12	5/8	8/12	10/10	0

La v1c (validateur de sources) n'apparaît pas dans ce tableau : son exactitude n'a pas été calculée. Ses scores RAGAS globaux sont proches de ceux de la v1b (faithfulness 0,711 contre 0,721 ; answer_correctness 0,550 contre 0,544).

Le critère de livraison, fixé avant l'évaluation de la v2e : au moins le niveau de la v2b dans chaque catégorie, sans erreur technique. La v2e le manque en multicritères (8/12 contre 10/12) : M02 cherche de nouveau le meilleur marqueur dans les 5 fiches récupérées (James Harden, 1 801 points, au lieu de Shai Gilgeous-Alexander, 2 485), M04 et M07 refusent au lieu d'interroger la base. **La v2b est donc la version livrée.** Les outils déterministes restent dans le dépôt, testés mais non branchés sur l'agent (tag `v2e-essai`).

Chaque version a corrigé un type d'erreur différent :

Version	Ce qu'elle a corrigé	Ce qui restait
v0	—	Excel aplati et découpé : la fiche de Harden était classée 91e sur 302 extraits ; les 4 multicritères exactes l'étaient par la célébrité des joueurs

Version	Ce qu'elle a corrigé	Ce qui restait
v1a	L'accès aux données : la fiche de Harden passe 1re sur 644	Le bon chiffre entouré d'inventions ; aucun refus
v1b	La fiabilité : refus, sources citées, réponses sobres	Erreurs de calcul ; classements impossibles sur 5 fiches
v2	Le calcul et le classement : multicritères de 1 à 8 sur 12	4 mauvais choix d'outil, 3 erreurs techniques
v2b	Le conflit entre documents et base ; les faux positifs du validateur	Questions mixtes : 2/8
v2c (essai)	Rien : règle de prompt et exemple few-shot ignorés	Rangs toujours faux
v2d	Rangs et comparaisons des questions mixtes : 2/8 → 6/8	Régression multicritères (10 → 7), limite d'appels contournée
v2e (essai)	Limite d'appels stricte, plus aucune erreur technique	Multicritères 8/12, mixtes 4/8 : non livrée

6.2 Le jeu de robustesse mixte (8 questions)

Question	v1b (sans SQL)	v2b (livrée)	v2c	v2d	v2e
MX01 — 3P% de Randle en saison (Reddit : 45 % en playoffs)	correct	correct	correct	correct	correct
MX02 — Cunningham a-t-il dépassé 35 % ? (35,6 %)	« non », faux	« non », faux	« non », faux	**« oui », correct**	**« oui », correct**
MX03 — duo d'Orlando cité sur Reddit et points par match	joueur inventé	refus à tort	joueur inventé	joueur inventé	joueur inventé
MX04 — rang de SGA aux lancers francs tentés (2e)	chiffre juste, rang non donné	« 1er », faux	« 1er », faux	**« 2e », correct**	**« 2e », correct**
MX05 — TS% du joueur proche de Reggie Miller (Kawhi Leonard)	refus à tort	mauvais joueur	mauvais joueur	mauvais joueur	mauvais joueur
MX06 — Edwards a-t-il le meilleur total à 3 points ? (oui)	comparaison non faite	« non », faux	« oui », juste par hasard	**« oui, rang 1 », correct**	« pas parmi les meilleurs », faux

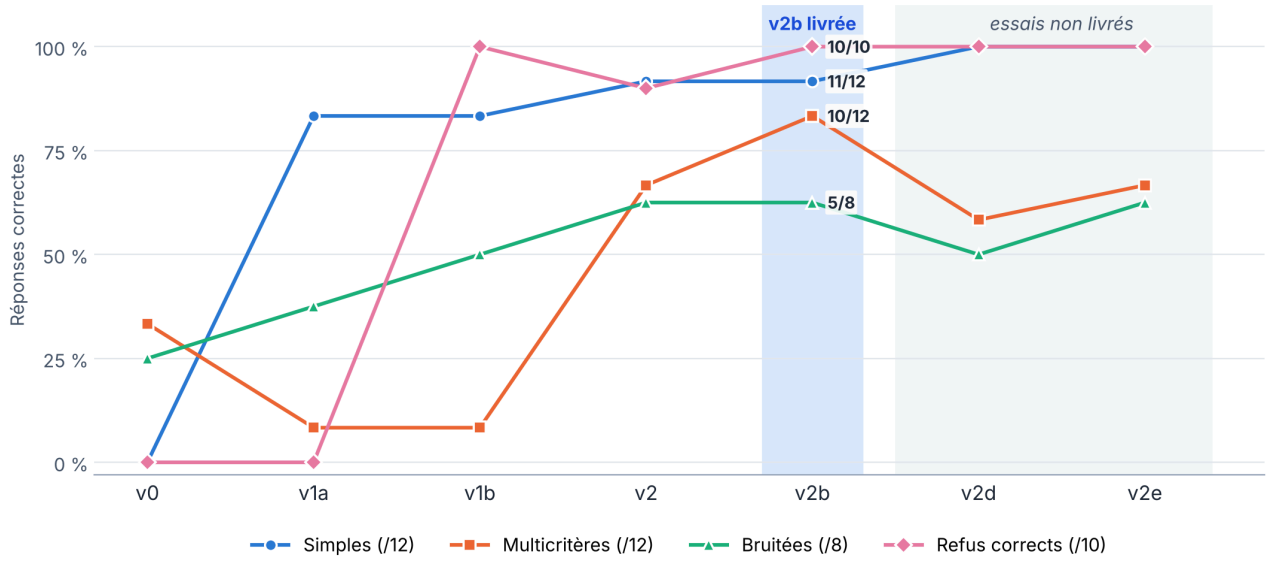
Question	v1b (sans SQL)	v2b (livrée)	v2c	v2d	v2e
MX07 — rang de Cunningham aux balles perdues (3e)	chiffre juste, rang non donné	erreur technique	« 1er », faux	**« 3e », correct**	« absent du classement », faux
MX08 — FG% de Reggie Miller cette saison (absent)	refus correct	refus correct	refus correct	refus correct	refus correct
Réponses correctes	**2/8**	**2/8**	**2/8**	**6/8**	**4/8**
Affirmations fausses	**2**	**4**	**5**	**2**	**4**

Sans outil SQL, la v1b échouait honnêtement (« je ne peux pas déterminer le rang »). Avec l'outil SQL génératif, la v2b et la v2c affirmaient un rang faux. Les outils déterministes de la v2d rendent les rangs et les comparaisons justes. La v2e retombe à 4/8 : sur MX06 et MX07, le rang n'est pas obtenu, et le modèle conclut quand même, à tort. Un tel écart entre deux versions qui ne diffèrent pas sur ces questions montre aussi la variabilité d'une exécution unique. Restent, dans toutes les versions, les deux questions qui demandent d'identifier un joueur dans le texte Reddit (MX03, MX05).

6.3 Les seuils atteints

En v2b, version livrée : questions simples au niveau cible (exactitude 0,917, faithfulness 0,903) ; multicritères au niveau cible (exactitude 0,944, faithfulness 0,860) ; bruitées en bêta pour la faithfulness (0,812) mais pas pour l'exactitude (0,625) ; textuelles non atteintes (faithfulness 0,689, answer_correctness 0,384) ; refus et erreurs au niveau cible. Le jeu mixte n'atteint ses seuils qu'avec les outils déterministes (v2d : exactitude 0,75 en bêta, faithfulness 0,888 en cible ; v2e : exactitude 0,68 en bêta).

6.4 Vue d'ensemble



Réponses correctes par catégorie et par version, jeu principal (exactitude déterministe et refus vérifiés, une exécution par version)

Le jeu principal progresse jusqu'à la v2b puis recule légèrement avec les outils déterministes, qui ne profitent qu'au jeu mixte, et de façon instable d'une exécution à l'autre.

7. Analyse critique

Les gains viennent de l'architecture (outil SQL, outils déterministes), pas du prompt ; et l'évaluation automatique ne suffit pas à les mesurer : RAGAS récompense l'assurance plutôt que la justesse, et son bruit dépasse les écarts entre versions proches.

7.1 Le passage du langage naturel au SQL

Le générateur SQL produit des requêtes **valides mais fausses**, qu'aucun garde-fou ne détecte : le garde-fou vérifie la sécurité d'une requête, pas sa justesse.

Erreur observée	Exemple	Conséquence
Filtre placé avant le classement	`WITH ... RANK() OVER (...) WHERE name LIKE '%gilgeous-alexander%'`	Le joueur est toujours « 1er » (MX04, MX06, MX07)
Tri sur la mauvaise colonne	Tirs tentés au lieu du pourcentage (B06) ; 20 joueurs ayant le plus tenté au lieu des meilleurs pourcentages (M05, v2d)	Classement faux, fidèlement restitué
Chiffres de saison présentés comme récents	« 5 derniers matchs » (H01, v2)	Source SQL légitime, réponse fausse

Erreur observée	Exemple	Conséquence
Pas de seuil de volume	Meilleur pourcentage sans minimum de tentatives	Un joueur à 2 sur 2 en tête, corrigé par un exemple few-shot
Requête hors sujet	Somme des balles perdues « des joueurs cités dans les documents » (MX07, v2c)	Pourcentage absurde (53,5 %)

7.2 Le prompt contre l'architecture

Trois tentatives sur le même défaut (le rang d'un joueur) donnent la leçon principale du projet :

1. **v2c, par le prompt** : une règle « demande le rang » et un exemple few-shot « classer puis filtrer ». L'agent demande bien le rang, mais le générateur SQL n'imite pas l'exemple : 2/8, comme la v2b. Une réponse juste (MX06) l'est par hasard.
2. **v2d, par l'architecture** : des outils au SQL écrit et testé à l'avance ; le modèle ne choisit que les paramètres. Questions mixtes : 6/8. Mais deux effets de bord sur le jeu principal : la règle 15 (« utilise rang_joueur et non interroger_statistiques ») a fait classer les joueurs des 5 fiches un par un au lieu d'interroger la base (M12 : 43 contres au lieu de Wembanyama, 175) ; et des appels parallèles ont contourné la limite (40 appels sur M05).
3. **v2e, fiabilisation** : règle 15 restreinte à un joueur déjà nommé, limite d'appels protégée par un verrou (testée sur 10 appels parallèles : 2 acceptés), noms abîmés des fiches nettoyés. Résultat : plus d'erreur technique, M12 corrigée, mais les multicritères restent à 8/12 (contre 10/12 en v2b) et le jeu mixte retombe à 4/8. Version non livrée.

Le modèle doit comprendre la question, le code doit faire le calcul. C'est la même logique que la vue `player_per_game`, qui retire déjà la division au modèle. Et une règle de prompt déteint au-delà de sa cible : la règle 13 a créé des précautions verbales jugées évasives par RAGAS, la règle 15 a détourné les classements. Chaque modification impose de réévaluer tout le jeu.

7.3 Les limites de RAGAS

- **Le bruit.** Entre deux exécutions identiques de la v0 : jusqu'à ±0,05 (answer_correctness), ±0,07 (faithfulness), ±0,12 (answer_relevancy) par catégorie, 0,51 sur une question. Les mêmes 3 questions, à 20 minutes d'intervalle : faithfulness 1,000 puis 0,778. Sur 8 à 12 questions par catégorie, une seule question change la moyenne de 0,06 à 0,12.
- **Il récompense l'assurance.** Sur le jeu mixte, la faithfulness monte de 0,44 (v1b) à 0,81 (v2c) pendant que les affirmations fausses passent de 2 à 5 : une réponse fausse, fidèle au résultat d'une requête fausse, est « fidèle au contexte ».
- **answer_relevancy** vaut 0 sur un refus par conception, et aussi sur des réponses justes formulées avec précaution (M06, M11, M12 en v2b), que le juge classe comme évasives : c'est l'essentiel de la baisse de −0,24 en multicritères entre v2 et v2b.
- **answer_correctness** a un plancher d'environ 0,2 et ne distingue pas 9,8 de 8,5.
- **context_precision** ne voit pas la voie SQL (0,14 à 0,18 en multicritères) : elle juge les fiches FAISS alors que la réponse vient de la base.
- **Les moyennes ne se comparent qu'à questions égales** : une version avec des questions en erreur est notée sur d'autres questions.

7.4 Les limites de l'exactitude et du validateur

- **L'exactitude vérifie la présence des éléments, pas la conclusion.** En v2b, MX02 (« 35,6 % ne dépasse pas 35 % ») et MX06 (« 324 n'est pas le meilleur total ») sont comptées exactes : les chiffres sont justes, les conclusions fausses. D'où la relecture manuelle du jeu mixte.
- **Le validateur garantit que les chiffres viennent des données, pas qu'ils répondent à la question** : Patty Mills (MX05) figurait dans les fiches récupérées, donc la source était « autorisée ». Trop strict sur la forme, il a aussi produit 3 faux positifs en v2 (« Reddit 2.pdf (Score: 77.1%) »), corrigés en v2b.

7.5 Sensibilité à un changement de corpus ou de modèle

Changement	Effet attendu	Ce qui le protège, ce qui ne le protège pas
Nouvelle saison, même format Excel	Faible	Validation Pydantic, base reconstruite par `setup_rag.py` ; mais références du jeu de test à recalculer (`scripts/build_testset.py`)
Nouvelles colonnes dans l'Excel	Moyen	Colonnes générées depuis le modèle Pydantic : il faut l'étendre, et décrire chaque colonne pour le LLM
Autres documents (articles, rapports de scouting)	Élevé	Le découpage par taille est adapté à Reddit de façon approximative ; les questions textuelles, déjà les plus faibles, seraient à réévaluer
Autre générateur	Élevé et imprévisible	Les règles et exemples ont été ajustés sur `ministral-3b` ; les outils déterministes, eux, ne dépendent pas du modèle. Réévaluation complète obligatoire
Autre juge	Élevé sur RAGAS	Les scores RAGAS changent d'échelle ; l'exactitude et les refus, déterministes, restent comparables

Le système est donc plus robuste là où le calcul est confié au code (base, vue, outils déterministes, exactitude) que là où il dépend du comportement du modèle (choix d'outil, restitution, lien entre sources).

7.6 Les limites du protocole

- **Une exécution par version** (sauf la v0) : les écarts inférieurs au bruit mesuré ne sont pas interprétés.
- **50 + 8 questions** : une question vaut 8 à 25 points de pourcentage dans sa catégorie ; aucune question ne porte sur une équipe.
- **Seuils du jeu principal fixés après observation** des premières versions.
- **Index vectoriel non reconstruit** depuis la v1a : il contient encore les noms bruts (« P,J, Washington »), contourné en v2e par le nettoyage dans les outils.

- **Reproductibilité** : le tag `v1b` ne contenait ni la liste complète de ses dépendances ni son index. Réévalué sur le jeu mixte avec l'environnement actuel et l'index d'origine, sans changer son code. Depuis, `pyproject.toml` et `uv.lock` sont commités avec le code qui en dépend, et l'index est versionné.

8. Suivi des performances dans le temps

Le suivi repose sur deux boucles : une **boucle continue** sur le trafic réel, avec Logfire, et une **boucle de régression** périodique, qui rejoue le jeu de test à chaque changement.

8.1 En continu, avec Logfire

Chaque question posée à l'API produit une trace (`answer` → `retrieval` → `prompt_building` → `agent run` → appels d'outils), avec ses attributs : sources et scores de la recherche, outil appelé et requête, `information_disponible`, nombre de sources citées, durée, erreur. À partir de ces attributs, un tableau de bord Logfire suit :

Indicateur	Ce qu'il détecte	Alerte suggérée
Taux d'erreurs techniques	Panne du modèle, validation échouée	Plus de 2 % des questions sur une journée
Taux de refus (`information_disponible = false`)	Hausse : données manquantes ou modèle trop prudent ; baisse brutale : inventions possibles	Écart de plus de 10 points à la moyenne de la semaine
Part des questions avec appel d'outil	Mauvais choix d'outil (la cause principale des échecs en v2)	Baisse sur les questions contenant « le plus », « meilleur », « rang »
Requêtes SQL en erreur ou corrigées	Dégradation du générateur SQL	Plus de 10 % de requêtes corrigées
Durée médiane et maximale	Lenteur, boucles d'appels	Médiane au-delà de 5 s
Score de la recherche (premier extrait)	Questions hors du corpus, dérive des sujets	Baisse durable de la médiane

Ces indicateurs ne disent pas si une réponse est juste : ils signalent quand regarder de plus près. Des captures d'écran de ces traces dans Logfire (une trace dépliée et la liste des traces) sont fournies en complément de ce rapport.

8.2 Périodiquement, avec le jeu de test

- **À chaque changement** (prompt, modèle, données, code) : `evaluate_ragas.py` sur les 50 + 8 questions, puis `comparer_versions.py` face à la version en production. Un changement n'est livré que s'il ne régresse sur aucune catégorie (exactitude, refus, erreurs) au-delà du bruit mesuré.
- **Chaque semaine**, même sans changement : une exécution planifiée détecte une dérive du fournisseur de modèles (une version servie différente donne d'autres réponses).
- **Historique** : chaque exécution est conservée dans `eval/results/<version>/` avec sa configuration, ce qui permet de tracer l'évolution des scores version par version.

8.3 Dans les rituels de SportSee

- **Point hebdomadaire** : le tableau de bord Logfire de la semaine et 5 à 10 traces tirées au hasard, relues par un analyste.
- **Revue avant livraison** : le bilan des seuils (`bilan_seuils.csv`) et la comparaison à questions égales.
- **Enrichir le jeu de test** avec les questions réelles mal traitées, repérées dans les traces : c'est ce qui garde l'évaluation représentative de l'usage.

9. Recommandations

L'assistant peut être ouvert en bêta aux coachs pour les questions chiffrées sur la saison, avec le suivi de la section 8, mais pas pour les questions sur les discussions Reddit ni pour celles qui croisent les deux sources.

9.1 À court terme

1. **Étendre les outils déterministes** aux autres calculs sensibles : écart entre deux joueurs (le M04 de la v2b, « combien de plus que le deuxième »), meilleurs joueurs d'une équipe, top N avec seuil de volume. C'est la correction qui a le mieux marché.
2. **Vérifier les joueurs cités face au texte** : si la réponse attribue un joueur à une discussion Reddit, son nom doit figurer dans un des extraits récupérés. Ce contrôle déterministe aurait bloqué les trois joueurs inventés (MX03, MX05).
3. **Contrôler les refus par le code** : un refus ne doit contenir aucun nombre présent dans les résultats SQL de la question.
4. **Exécuter chaque version au moins trois fois** et ne retenir que les écarts supérieurs au bruit mesuré.
5. **Piloter avec l'exactitude, les refus vérifiés et une relecture des conclusions** ; garder RAGAS pour le diagnostic, et retirer `answer_relevancy` et `context_precision` du tableau de bord tant que leurs angles morts demeurent.

9.2 Pour les questions textuelles et mixtes

C'est le principal chantier restant (answer_correctness 0,38 en v2b, sous le seuil bêta de 0,50). Pistes, à évaluer une par une : découper les PDF par fil de discussion plutôt que par taille ; récupérer plus de 5 extraits ; recherche hybride (mots-clés et vecteurs) ; reconstruire l'index avec les noms nettoyés ; pour les questions mixtes, un pipeline en deux étapes explicites (identifier le joueur dans le texte, puis interroger la base).

9.3 Pour la mise en production

- **Un générateur plus grand**, si l'abonnement le permet, pour le choix d'outil et la restitution ; le juge devrait alors rester un modèle différent du générateur.
- **Des données match par match** pour alimenter la table `matches` déjà prévue et rendre possibles les questions sur une période.
- **Le suivi de la section 8** : tableau de bord et alertes Logfire, évaluation planifiée, revue hebdomadaire.
- **Une source de données corrigée** : remonter au fournisseur les anomalies du journal d'audit.

10. Conclusion

Chaque version a corrigé un type d'erreur différent : la v1a l'accès aux données, la v1b la fiabilité (refus, sources), la v2 le calcul et le classement, la v2d les rangs et les comparaisons. À chaque fois, le gain est venu d'un déplacement du travail du modèle vers le code : une fiche par joueur plutôt qu'un tableau découpé, une base qui trie et divise, des outils au SQL écrit à l'avance. Les règles de prompt, elles, ont souvent été ignorées ou ont déteint au-delà de leur cible.

Le second enseignement porte sur l'évaluation elle-même. Aucune métrique automatique ne suffit seule : RAGAS a noté favorablement des réponses fausses et assurées, pénalisé des refus honnêtes et des réponses justes, et son bruit dépasse les écarts entre versions proches ; l'exactitude déterministe, elle, ne vérifie pas les conclusions. Évaluer un assistant de ce type demande plusieurs mesures complémentaires, une comparaison à questions égales, une mesure du bruit, un jeu de robustesse et une relecture manuelle des cas limites.

Annexes

A. Scores RAGAS par version (jeu principal)

Moyennes globales sur les 50 questions (v0 : première exécution). À comparer avec prudence : les ensembles notés diffèrent (questions en erreur) et les catégories de refus faussent answer_relevancy.

Métrique	v0	v1a	v1b	v2	v2b	v2d
faithfulness	0,216	0,459	0,721	0,822	0,775	0,845
context_precision	0,194	0,449	0,476	0,509	0,503	0,486
context_recall	0,313	0,640	0,590	0,762	0,750	0,736
answer_correctness	0,257	0,274	0,544	0,631	0,657	0,616

Par catégorie à réponse (faithfulness / answer_correctness) :

Catégorie	v1b	v2	v2b	v2d
Simples	0,842 / 0,789	0,875 / 0,851	0,903 / 0,797	0,944 / 0,814
Bruitées	0,658 / 0,470	1,000 / 0,528	0,812 / 0,611	0,875 / 0,443
Multicritères	0,640 / 0,314	0,789 / 0,636	0,860 / 0,683	0,843 / 0,572
Textuelles	0,710 / 0,422	0,760 / 0,374 (5 notées)	0,689 / 0,384	0,718 / 0,465

Jeu mixte (faithfulness / answer_correctness) : v1b 0,510 / 0,459 ; v2b 0,671 / 0,475 ; v2c 0,783 / 0,439 ; v2d 0,888 / 0,646.

B. Règles ajoutées au prompt de l'agent

Le prompt reprend les 6 règles du prompt v1 (orienté coachs, `rag_pipeline.py`) et ajoute :

Règle	Contenu	Problème traité	Version
7	Question chiffrée : appeler l'outil avant de répondre	Faire utiliser l'outil	v2
8	Les chiffres de l'outil priment ; ne pas les recalculer	Erreurs de calcul	v2
9	Citer « Base de données NBA (SQL) »	Traçabilité	v2
10	Aucune ligne ou erreur : information non disponible	Inventions	v2
11	Données de saison entière ; période précise : non disponible	Chiffres de saison présentés comme récents	v2
12	Chiffres donnés : information disponible et sources non vides	Drapeau incohérent	v2
13	Documents = extrait ; superlatif ou moyenne par match : outil obligatoire	Mauvais choix d'outil	v2b
14	Résultats de l'outil = contexte valide	Résultats SQL rejetés	v2b
15	Rang ou seuil d'un joueur déjà nommé : outils déterministes ; classements : outil SQL	Rangs et comparaisons faux	v2d, restreinte en v2e

C. Exemples few-shot de l'outil SQL et outils déterministes

Aucun exemple ne reprend une question du jeu de test : ils enseignent des motifs, pas des réponses.

Question d'exemple	Motif enseigné
Interceptions d'Anthony Edwards	Valeur simple, jointure, recherche sans accents
Passes par match pour Doncic	Vue des moyennes par match
5 meilleurs intercepteurs (au moins 50 matchs)	Filtre, tri, limite
True shooting supérieur à 65 % (au moins 500 tirs)	Pourcentage exprimé de 0 à 100
Meilleur passeur par match des Nuggets	Filtre par nom d'équipe
Durant contre LeBron James (points par match)	Comparaison de deux joueurs
Équipe avec le plus de joueurs à 1 000 points	Agrégation (GROUP BY)
Score Lakers - Celtics du 25 décembre	Table `matches` (vide)
Meilleur pourcentage aux tirs des Lakers	Seuil de volume implicite

Les outils déterministes (`utils/outils_classement.py`) utilisent une requête fixe : `RANK() OVER (ORDER BY statistique)` calculé sur tous les joueurs ayant au moins `min_matchs` matchs, **puis** filtré sur le joueur demandé.

D. Fichiers et artefacts

Élément	Rôle
`setup_rag.py`, `api.py`	Mise en place en un lancement ; API FastAPI
`rag_pipeline.py`	Recherche, agent Pydantic AI, fonction `answer()`
`indexer.py`, `utils/data_loader.py`, `utils/vector_store.py`	Préparation des PDF et index FAISS
`load_excel_to_db.py`, `utils/db_schema.py`, `utils/schemas.py`	Chargement validé, schéma de la base, modèles Pydantic
`sql_tool.py`, `utils/agent_outils.py`, `utils/outils_classement.py`	Outils de l'agent : SQL génératif et outils déterministes
`evaluate_ragas.py`, `utils/bilan_seuils.py`, `utils/exactitude.py`	Évaluation, seuils, exactitude
`scripts/`	Comparaison de versions, refus, requêtes SQL, origine des réponses, diagnostics
`tests_rag/test_outils_classement.py`	Tests déterministes des outils
`eval/testset.json`, `eval/testset_mixte.json`, `eval/elements_attendus*.json`	Jeux de test et éléments attendus
`eval/results/<version>/`	Réponses, scores, configuration et journal de chaque évaluation
`docs/audit/anomalies_donnees.csv`	Journal qualité des données
`pyproject.toml`, `uv.lock`, `requirements.txt`	Environnement reproductible